Trace Cache Sampling Filter

Michael Behar, Avinoam Kolodny, Avi Mendelson

Abstract—**This paper presents a new technique for efficient usage of small trace caches. A trace cache can significantly increase the performance of wide out-of-order processors, but to be effective, the size of the trace cache should be large.**

Power and timing considerations indicate that a small trace cache is desirable, with special mechanisms to increase its effectiveness despite the limited size. Hence several authors have proposed various filtering methods to select "good traces" for keeping in the trace cache, from among the general population of traces.

This paper presents a new filtering technique, which is based on sampling. Our new technique suggests that instead of building all the traces and trying to select the good ones among them, it is more efficient to make a preliminary selection of traces. This selection is based on a random sampling approach.

We show that the Sampling Filter improves trace cache and overall system performance, while reducing power dissipation. The Sampling Filter reduces admission of traces that are not used prior to their eviction from the cache, and prolongs the percentage of time a trace is in its live phase during its stay in the cache. Moreover, the Sampling Filter reduces duplication between the trace cache and the instruction cache and thus reduces the overall misses in the first level of cache hierarchy.

Index Terms—**Trace cache, Filtering, Performance, and Power.**

1 Introduction

The effectiveness of a single threaded out-of-order processor is strongly dependent on the average number of useful instructions it can fetch in every cycle. Trace caches are very effective in serving this goal by storing instructions in their dynamic order rather than in their static order [10] [15] [16]. Thus, a trace cache has two major merits: it contains instructions from different basic blocks that would normally require several accesses to the instruction cache in order to be fetched, and it contains only useful instructions (contrary to an instruction cache line that is fetched but might include instructions that the processor doesn't require). The trace cache was found to be very effective if size and power are not limited. However, relatively small caches which are practical in term of access time and power consumption, are very vulnerable in terms of memory efficiency, because of several problems [11]: the same basic blocks can appear in different traces (duplication), some traces don't contain the maximum number of instructions (fragmentation), and the requested block might reside inside a trace and thus become inaccessible (Indexability). Thus, the key for efficient usage of relatively small trace caches is either to change the way the trace is constructed; i.e., the use of basic block traces [1], or to keep only the most valuable traces inside the cache and thus avoid their trashing by less valuable traces. This work will focus on the second option since the logic required to construct the traces out of a block-based cache is very costly in term of power.

Filtering (selective admission of incoming traces to the cache) has already been proposed as a way to increase the usefulness of a limited size trace cache. In [12] it was proposed to store only traces containing taken branches, which cannot be fetched in one access from the instruction cache. In [13] it was proposed to filter traces based on their usage. The trace cache is divided into two blocks: the Filter trace Cache (FTC) and the Main Trace Cache (MTC). All traces are written to the FTC, but only traces that have been proven to be "useful" are inserted to the MTC. The success of this filtering method is based on the important observation presented there [13], that most traces that are built and inserted to the trace cache are rarely used before eviction, and that most of the instructions the processor executes come from a small set of traces ("hot traces"). In [8] it was proposed to use profiling in order to filter out traces that are less frequent and show little time locality.

This paper presents a new class of trace filtering techniques, which is based on statistical sampling of traces. This class of filters aims to improve the quality of the traces residing in a small trace cache, while reducing the power dissipation needed for maintaining the filter's bookkeeping. The paper presents and analyzes the performance and the power of a basic Sampling Filter (SF) and an enhanced version of it, and compares its performance and power with a regular trace cache and with the FTC-MTC organization.

The rest of the paper is organized as follows: In Section 2 the simulation environment and the characterization of traces are discussed; Section 3 describes the sampling filter architecture and compares it with the regular trace cache and with the FTC-MTC organization. In Section 4 the usage of the Sampling Filter with the FTC-MTC organization is demonstrated and Section 5 concludes and proposes related ideas for future studies.

2 Simulation Environment and Basic Observations

This section presents the simulation environment we used, and some of the basic observations that our new proposed technique are based upon.

2.1 The simulation framework

The performance numbers presented below are based on an extended version of the sim-outorder simulator from the SimpleScalar tools set 3.0d [1] that was augmented with a detailed model of the trace cache that includes the impact of wrong path prediction and recovery, along with the simulation of the proposed filter mechanism and with a next trace predictor [6]. The power numbers presented in this paper were computed by an extended version of the Wattch [1] simulator